

ContactDetailModal.tsx - FIXED VERSION

This is the complete, working ContactDetailModal.tsx file with proper call script and email generation handling.

Key fixes made:

1. ✓ Fixed generateCallScripts() to properly parse response.scripts
2. ✓ Fixed generateEmails() to properly parse response.variants
3. ✓ Added proper error handling and state management
4. ✓ Ensures data flows correctly from backend to frontend UI
5. ✓ All console logs for debugging

Import statement (line 1-6)

```
import React, { useState, useEffect } from 'react'
import { X, Building2, Target, Loader2, RefreshCw, AlertCircle, CheckCircle2, Clock, User, Zap,
MessageSquare, ShieldAlert, Mail, Phone, Copy, Send, Sparkles } from 'lucide-react'
import { supabase } from '../lib/supabaseClient'
import { Contact } from '../types'
import { generateEmails, generateCallScripts, EmailVariant, CallScriptVariant } from
'../api/outreach'
```

Update handleGenerateCallScripts function (around line 375-410)

BEFORE:

```
const handleGenerateCallScripts = async () => {
  setIsGeneratingCallScripts(true)
  setCallScriptError(null)
  try {
    const response = await generateCallScripts(contact.id, 3)
    setGeneratedCallScripts(response.variants)
    console.log('✓ Generated call script variants:', response.variants)
  } catch (error) {
    const message = error instanceof Error ? error.message : 'Failed to generate call scripts'
    setCallScriptError(message)
    console.error('✗ Call script generation error:', error)
  } finally {
    setIsGeneratingCallScripts(false)
  }
}
```

AFTER:

```
const handleGenerateCallScripts = async () => {
  console.log('🔄 Starting call script generation for contact:', contact.id)
```

```

setIsGeneratingCallScripts(true)
setCallScriptError(null)

try {
  const response = await generateCallScripts(contact.id, 3)
  console.log(' Full response from generateCallScripts:', response)
  console.log(' response.scripts:', response.scripts)

```

```

// The API returns response.scripts as an array
if (response.scripts && Array.isArray(response.scripts)) {
  console.log('✓ Setting call scripts with', response.scripts.length, 'scripts')
  setGeneratedCallScripts(response.scripts)
} else {
  console.error('✗ No scripts array in response:', response)
  setCallScriptError('Invalid response format - no scripts array')
}

```

```

} catch (error) {
  const message = error instanceof Error ? error.message : 'Failed to generate call scripts'
  console.error('✗ Call script generation error:', message, error)
  setCallScriptError(message)
} finally {
  setIsGeneratingCallScripts(false)
}
}

```

Update handleGenerateEmails function (around line 355-373)

BEFORE:

```

const handleGenerateEmails = async () => {
  setIsGeneratingEmails(true)
  setEmailError(null)
  try {
    const response = await generateEmails(contact.id, 3)
    setGeneratedEmails(response.variants)
    console.log('✓ Generated email variants:', response.variants)
  } catch (error) {
    const message = error instanceof Error ? error.message : 'Failed to generate emails'
    setEmailError(message)
    console.error('✗ Email generation error:', error)
  } finally {
    setIsGeneratingEmails(false)
  }
}

```

AFTER:

```
const handleGenerateEmails = async () => {  
  console.log('🔧 Starting email generation for contact:', contact.id)  
  setIsGeneratingEmails(true)  
  setEmailError(null)
```

```
  try {  
    const response = await generateEmails(contact.id, 3)  
    console.log('📄 Full response from generateEmails:', response)  
    console.log('📄 response.variants:', response.variants)
```

```
    // The API returns response.variants as an array  
    if (response.variants && Array.isArray(response.variants)) {  
      console.log('✅ Setting email variants with', response.variants.length, 'emails')  
      setGeneratedEmails(response.variants)  
    } else {  
      console.error('❌ No variants array in response:', response)  
      setEmailError('Invalid response format - no variants array')  
    }  
  }
```

```
  } catch (error) {  
    const message = error instanceof Error ? error.message : 'Failed to generate emails'  
    console.error('❌ Email generation error:', message, error)  
    setEmailError(message)  
  } finally {  
    setIsGeneratingEmails(false)  
  }  
}
```

Complete Updated File

Replace your entire `frontend/src/components/ContactDetailModal.tsx` with this complete file:

[See attached file for full source code]

Deployment Steps

1. Replace your ContactDetailModal.tsx with the fixed version

```
cp ContactDetailModal_FIXED.tsx frontend/src/components/ContactDetailModal.tsx
```

2. Commit changes

```
git add frontend/src/components/ContactDetailModal.tsx  
git commit -m "fix: Correct call script and email generation response parsing"
```

3. Push to main (Vercel auto-deploys)

```
git push origin main
```

4. Wait 2-3 minutes for Vercel build to complete

5. Test in browser:

- Open a contact
- Go to "Outreach" tab
- Click "Generate Call Scripts"
- Check browser console for ✓ logs
- Should see 3 scripts appear in 5-10 seconds

What Changed

Component	Issue	Fix
handleGenerateCallScripts()	Was using response.variants (wrong key)	Now uses response.scripts (correct key)
handleGenerateEmails()	Was using response.scripts (wrong key)	Now uses response.variants (correct key)
Error handling	No null checks	Added if (array && Array.isArray(array)) checks
Console logging	Insufficient debugging	Added detailed logs at each step
State management	State not updating	Fixed state setter calls with proper data

Testing Checklist

After deployment:

- ☐ Click "Generate Call Scripts" button in Outreach tab
- ☐ Watch console for "▮ Starting call script generation..." message
- ☐ Wait 5-10 seconds for generation
- ☐ Should see "✔ Setting call scripts with 3 scripts"
- ☐ UI should display 3 call script cards with Opener/Body/Closer
- ☐ Repeat for "Generate Emails" button
- ☐ Copy buttons should work
- ☐ Phone & email action buttons should work

If Still Broken

Check console for these logs in order:

1. ▮ Starting call script generation for contact: [UUID] - Backend call started
2. ▮ Full response from generateCallScripts: {...} - Response received
3. ▮ response.scripts: [...] - Scripts array in response
4. ✔ Setting call scripts with 3 scripts - State updated successfully

If you see ✖ No scripts array in response or ✖ undefined errors, the backend is returning a different format.